

■ Lab 10 — Data Sources

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Utilisé un **data source** pour récupérer l'AMI Amazon Linux la plus récente
- Compris la différence entre **Push** (créer) et **Pull** (lire)
- Utilisé `data...` dans vos ressources

■ Étape 1 — Créer les fichiers

```
cd labs/lab10-datasources
```

``backend.tf``

```
terraform {
  backend "s3" {
    bucket      = "tf-training-<votre-prenom>-982908300187"
    key         = "terraform.tfstate"
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
```

■■ Remplacez `` par votre prénom. Ex : `tf-training-alice-982908300187`

``versions.tf``

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

``provider.tf``

```
provider "aws" {  
  region = var.region  
}
```

`variables.tf`

```
variable "username" {  
  description = "Votre prenom"  
  type        = string  
}  
  
variable "region" {  
  description = "Region AWS"  
  type        = string  
  default     = "eu-west-1"  
}
```

`locals.tf`

```
locals {  
  prefix = "lab10-${var.username}"  
  
  common_tags = {  
    Lab      = "lab10"  
    Username = var.username  
    ManagedBy = "Terraform"  
  }  
}
```

`main.tf`

```
# Data source : récupère l'AMI Amazon Linux 2023 la plus récente
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# Data source : récupère les informations du compte AWS courant
data "aws_caller_identity" "current" {}

# Data source : récupère la région courante
data "aws_region" "current" {}

resource "aws_security_group" "lab10_sg" {
  name          = "${local.prefix}-sg"
  description   = "Security group Lab 10 - ${var.username}"

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-sg"
  })
}

resource "aws_instance" "lab10_instance" {
  # Utilisation du data source pour l'AMI
  ami              = data.aws_ami.amazon_linux.id
  instance_type    = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab10_sg.id]

  tags = merge(local.common_tags, {
    Name      = "${local.prefix}-instance"
    AmiName   = data.aws_ami.amazon_linux.name
  })
}
```

■ ****Push**** → `resource` crée une ressource sur AWS.

****Pull**** → `data` lit une ressource existante sur AWS sans la créer.

On référence un data source avec `data...`.

outputs.tf

```
output "ami_id" {
  description = "ID de l'AMI récupérée via data source"
  value       = data.aws_ami.amazon_linux.id
}

output "ami_name" {
  description = "Nom de l'AMI"
  value       = data.aws_ami.amazon_linux.name
}

output "ami_creation_date" {
  description = "Date de création de l'AMI"
  value       = data.aws_ami.amazon_linux.creation_date
}

output "account_id" {
  description = "ID du compte AWS"
  value       = data.aws_caller_identity.current.account_id
}

output "current_region" {
  description = "Région courante"
  value       = data.aws_region.current.name
}

output "instance_id" {
  description = "ID de l'instance"
  value       = aws_instance.lab10_instance.id
}
```

■ Étape 2 — Déployer et inspecter les data sources

```
terraform init
terraform apply -var="username=<votre-prenom>"
```

Vérifiez les outputs :

```
terraform output ami_id
terraform output ami_name
terraform output ami_creation_date
terraform output account_id
terraform output current_region
```

■ L'AMI récupérée est toujours la **plus récente** — plus besoin de hardcoder un ID d'AMI qui change selon les régions et les mises à jour AWS.

■ Étape 3 — Inspecter le data source dans le state

```
terraform state list
terraform state show data.aws_ami.amazon_linux
```

■ Les data sources apparaissent dans le state avec le préfixe `data.` mais Terraform ne les gère pas — il les lit uniquement.

■ Étape 4 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
```

■ Validation du Lab

#	Critère	Validé
1	`ami_id` retourne un ID AMI valide (format `ami-xxx`)	■
2	`ami_name` contient `al2023`	■
3	`account_id` retourne `982908300187`	■
4	`current_region` retourne `eu-west-1`	■
5	`terraform state show data.aws_ami.amazon_linux` affiche les détails	■
6	`terraform destroy` supprime les ressources (pas les data sources)	■

■ ****Fin du Lab 10 — Vos AMIs sont toujours à jour grâce aux data sources !****

Le Lab 11 introduit les ****provisioners**** pour exécuter des scripts sur les instances.